PipeMind

Autonomous ML Lifecycle Management

Project Documentation

DEPI Round 5

Agentic AI and Generative AI System Developement

Team Members:

Habiba Sherif Mahmoud
Hamass Nagieb Mortada
Ahmed Ibrahim Mahmoud
Alshimaa Sobhy Saad
Esraa Elsayed Amer
Aya Anwar Mahmoud

1. Introduction

1.1 Project Overview

PipeMind is a system that manages the machine learning (ML) lifecycle autonomously. The user provides an objective and a dataset, and the system takes them to a verified model that is ready for deployment.

PipeMind does not only recommend ML actions. It reasons about the task, executes experiments, evaluates the results, verifies them with an independent reviewer, and adapts its plan when the results are not good enough.

1.2 The Problem

ML development involves repetitive tasks, continuous experimentation, and frequent human decisions such as “what should I try next?”. This leads to high effort, long development cycles, avoidable errors, and slow deployment.

1.3 Business Objectives

- Reduce human effort in building ML models.
- Reduce the time needed to reach a working model.
- Improve experimentation efficiency.
- Improve model reliability.
- Improve reproducibility of results.

1.4 How PipeMind Works

Figure 1 shows the journey at a high level. The system first assesses the user's objective and dataset, then plans and executes an ML experiment. An independent Critic reviews the result. If it is rejected, the system returns to planning. If it is accepted, the user approves it before the model is deployed. Experiment tracking runs alongside every iteration.

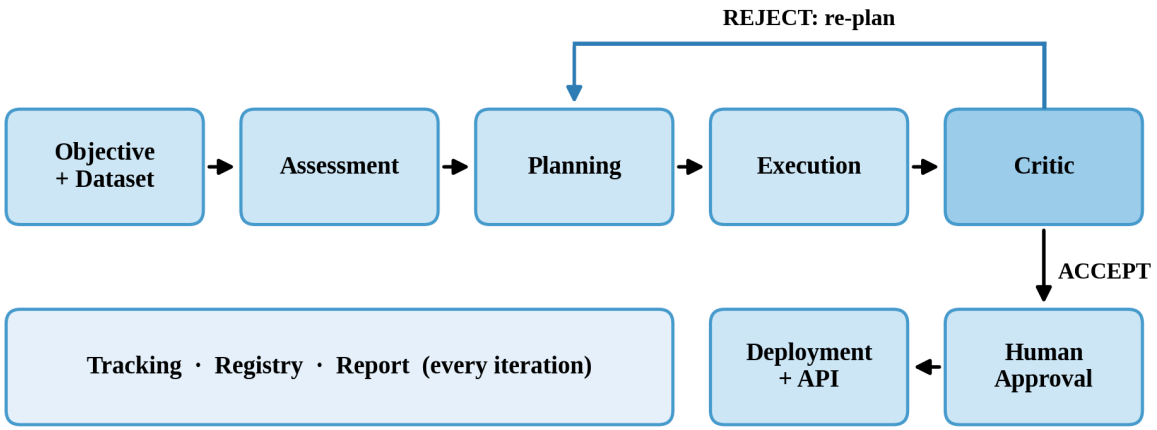


Figure 1. High-level flow of PipeMind

1.5 Technical Tools

LangGraph — Stateful Multi-Agent Workflow

- Defines the overall agent workflow as a graph.
- Each agent is represented as a node.
- State is shared across the workflow.
- Supports conditional routing and iterative loops.
- Enables Human-in-the-Loop decisions.
- Maintains experiment history and execution state.

Why LangGraph?

Our workflow is dynamic, stateful, and iterative not a fixed pipeline.

2. Project Planning & Management

2.1 Team & Roles

The project is divided into six components, and each team member is responsible for one of them.

Member	Responsibility
M1	Task Assessment, Orchestration and Integration
M2	Planning
M3	Execution
M4	Critic
M5	Tracking, Registry and Reports
M6	Packaging and Deployment

2.2 How We Work

- **Ownership:** each member owns one component of the system.
- **Integration:** M1 combines the components and tests them end to end.
- **Process:** work moves in stages, from analysis to testing.

3. Literature Review

3.1 Existing Solutions

Several kinds of solutions already address parts of the ML lifecycle. They fall into three groups.

Type	Examples	What they do
Cloud AutoML platforms	Google Vertex AI, Amazon SageMaker Autopilot, DataRobot	Automate model building, tuning and deployment, usually inside a managed cloud or enterprise platform.
Experiment tracking tools	MLflow	Record experiments and manage model versions. They record and organize; they do not plan or execute experiments.
Agent-based research	AutoML-Agent	A multi-agent framework built on large language models that covers the ML pipeline from the task description to deployment, with verification across several stages [1].

3.2 Where PipeMind Fits

Existing solutions address different aspects of the machine learning lifecycle. AutoML platforms automate model development and optimization, while tools such as MLflow support experiment tracking and model lifecycle management. Research frameworks such as AutoML-Agent explore the use of LLM-based agents to automate multiple stages of the ML pipeline.

PipeMind builds on these existing approaches by proposing an agent-based workflow that starts with the user's objective and uploaded dataset. A Planner Agent proposes ML experiments, an execution component runs them, and an independent Critic Agent reviews the results and provides a justification for its decision. Based on the evaluation outcome, the system can return to the planning stage for further experiments, subject to a defined attempt limit. The user remains involved in key decisions, and deployment proceeds only after the required review and user approval.

Rather than replacing existing AutoML platforms or experiment tracking tools, PipeMind aims to integrate goal-driven experiment planning, iterative evaluation, experiment tracking, and human oversight into a single workflow. Its focus is on coordinating these capabilities through multiple agents while keeping the experimentation and deployment process transparent and under the user's control.

4. Requirements Gathering

4.1 Functional Requirements

The system shall:

- **FR-1:** Extract metadata from the dataset uploaded by the user, such as column names, data types and number of rows.
- **FR-2:** Analyze the user's objective, identify the task type and the target, and ask the user for clarification when the objective is unclear instead of guessing.
- **FR-3:** Plan an ML experiment suited to the objective and the data, including preprocessing and model choices.
- **FR-4:** Execute the plan, including data preparation, model training and performance measurement.
- **FR-5:** Review each experiment result independently and decide ACCEPT or REJECT with a justification.
- **FR-6:** Present an accepted result to the user and wait for the user's approval before any further step.
- **FR-7:** Return to planning and run a new experiment when a result is rejected, up to a defined maximum number of attempts.
- **FR-8:** Deploy the approved model as a service that exposes a prediction API.
- **FR-9:** Track every experiment (parameters, metrics and artifacts), register the approved model with a version, and produce a final report.

4.2 Non-Functional Requirements

- **Reliability:** No model is delivered without a Critic review and the user's approval.
- **Reproducibility:** Every experiment is logged so that it can be re-run and its result checked.
- **Transparency:** Every decision, such as a rejection, comes with a clear reason the user can follow.
- **Controlled autonomy:** The system asks the user when it is unsure, and the improvement loop has a limit.